



บทที่ 6

การมีได้หลากหลายรูปแบบ

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรรยาโรจน์

การเขียนโปรแกรมเชิงวัตถุ

การห่อหุ้ม
Encapsulation

การสืบทอด
Inheritance

การมีได้
หลากหลายรูปแบบ
Polymorphism

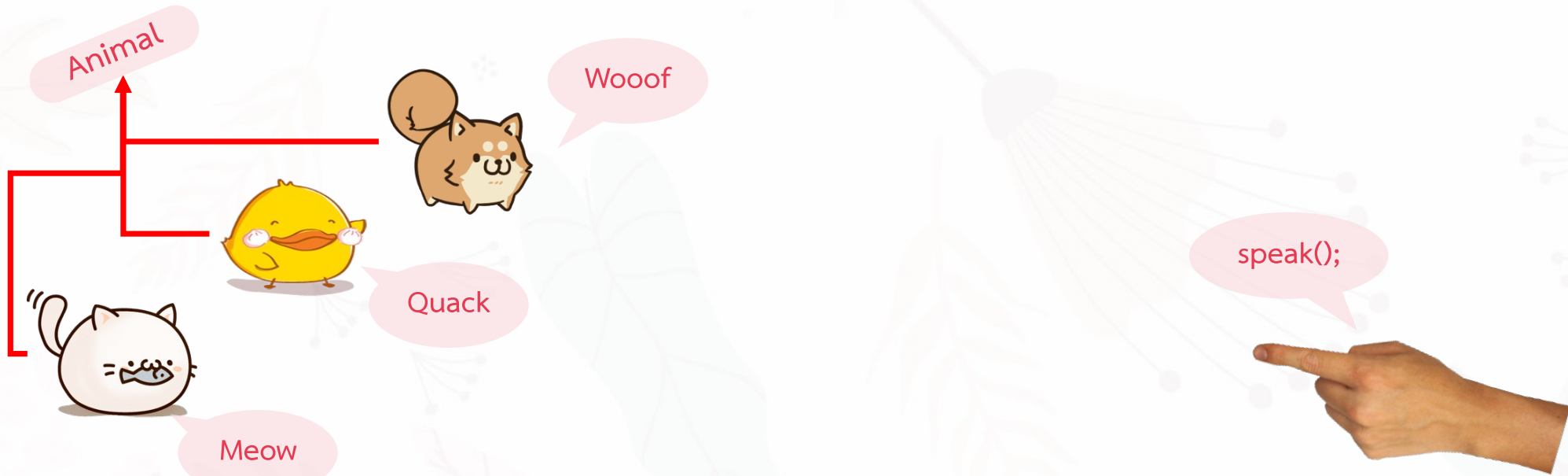
การเขียนโปรแกรมเชิงวัตถุ

การห่อหุ้ม
Encapsulation

การสืบทอด
Inheritance

การมีได้
หลากหลายรูปแบบ
Polymorphism

การมีหลากหลายรูปแบบ (Polymorphism)



Polymorphism มีรากศัพท์มาจากภาษากรีกคำว่า “poly” และ “morphs” โดยที่ "Poly" หมายถึงมากมาย และ "Morphs" หมายถึงรูปแบบ ดังนั้น จึงมีความหมายว่ารูปแบบนับไม่ถ้วนหรือรูปแบบที่หลากหลาย

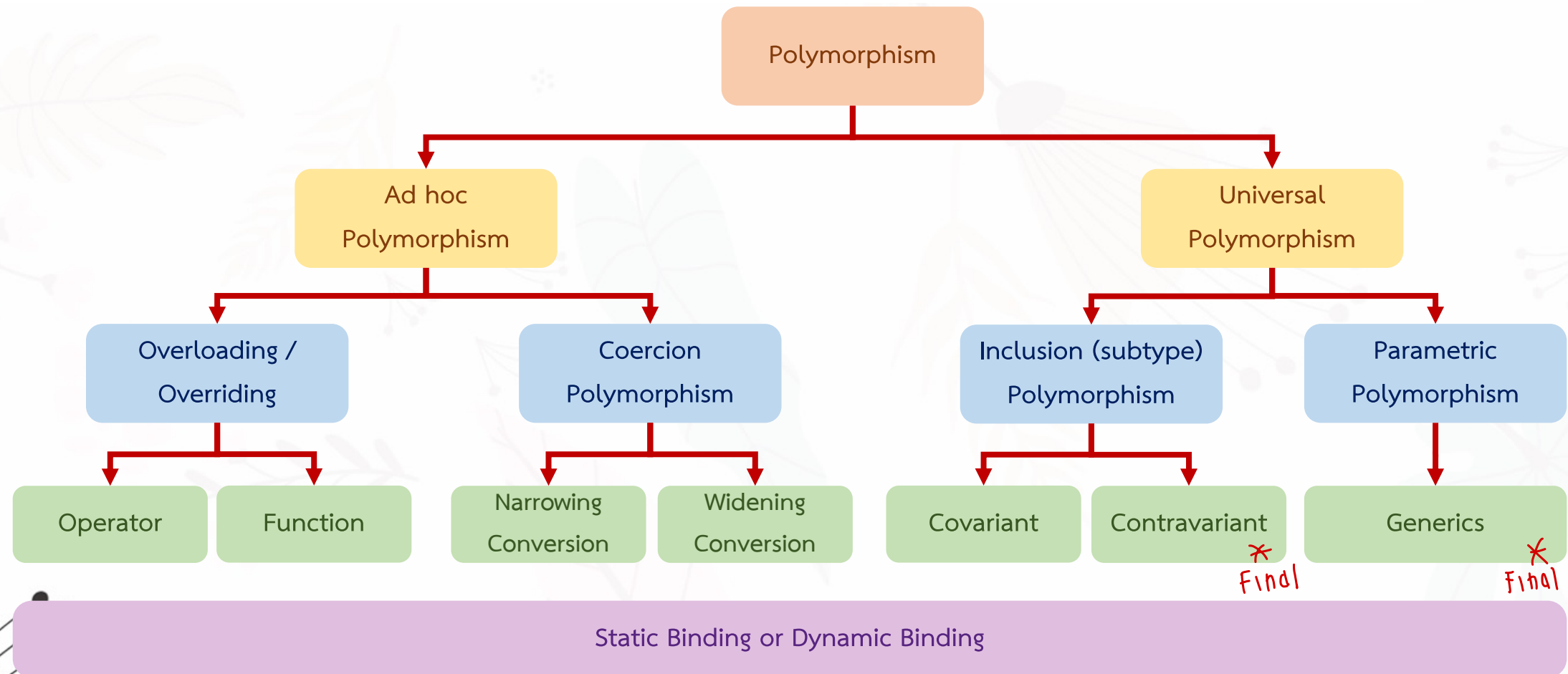
การมีหลากหลายรูปแบบ (Polymorphism)



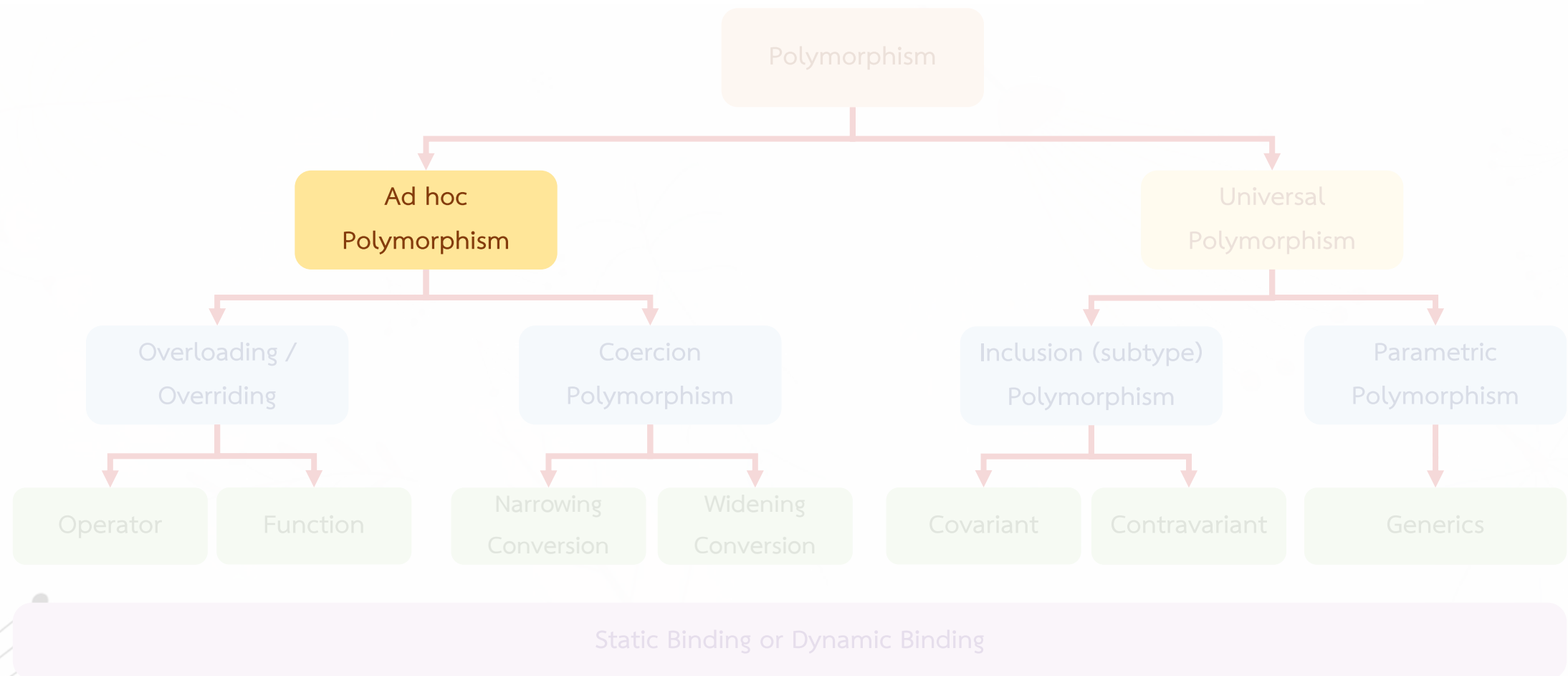
ระยะเวลาที่อยู่	บริบท
บ้าน	ลูก
โรงเรียน	นักเรียน
ร้านอาหาร	ลูกค้า
รถโดยสาร	ผู้โดยสาร

สมมติว่าถ้านักศึกษาอยู่ที่โรงเรียนเวลานั้นนักศึกษาทำตัวเหมือนนักเรียน เวล่านักศึกษาไปตลาดเวลานั้นนักศึกษาทำตัวเหมือนลูกค้า เวล่านักศึกษาอยู่บ้านเวลานั้นนักศึกษาทำตัวเหมือนลูกชายหรือลูกสาว ดังนั้น จะพบ**บุคคลหนึ่งแสดงพฤติกรรมแตกต่างกันในสถานการณ์แตกต่างกัน**

การมีหลากหลายรูปแบบ (Polymorphism)



การมีหลากหลายรูปแบบ (Polymorphism)



Ad hoc Polymorphism



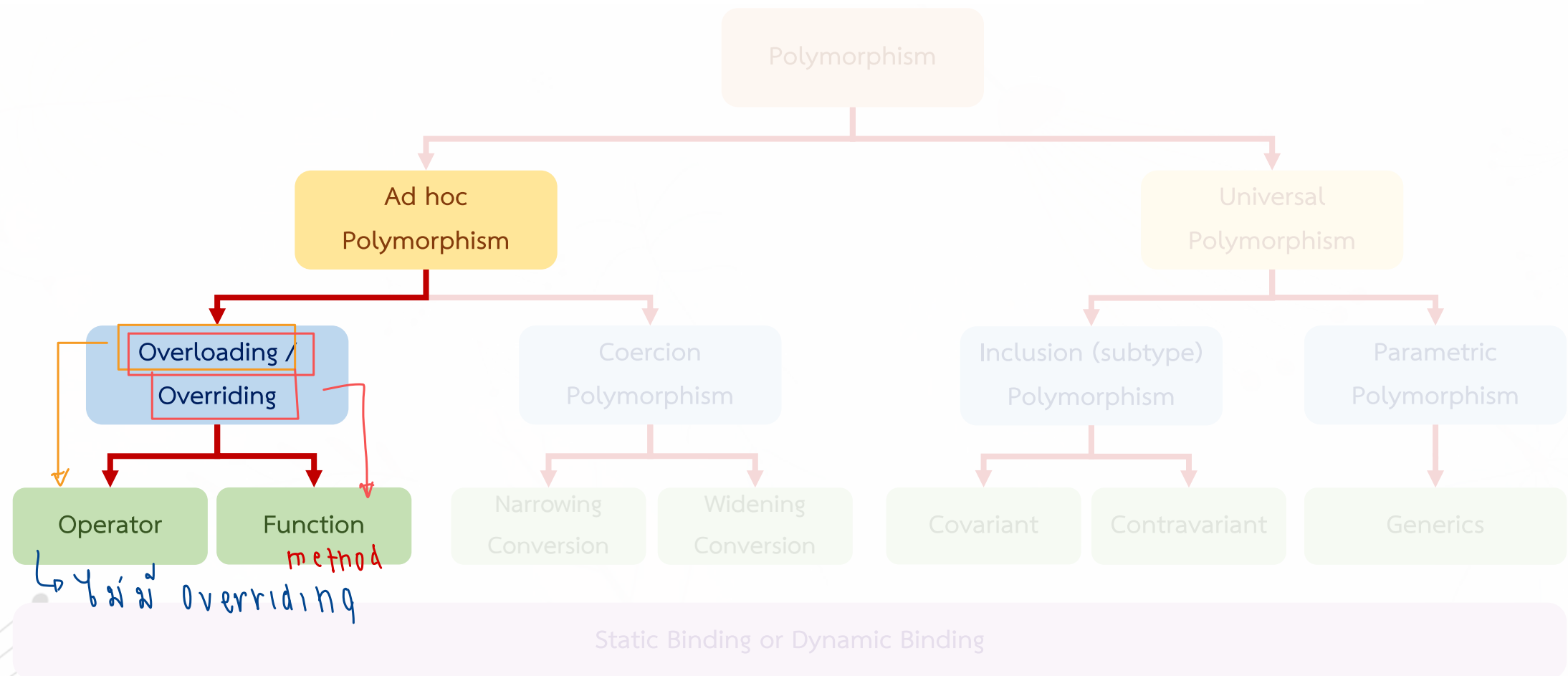
(Photo: courtesy of AlanTuring.net)

Christopher Strachey นักวิจัยทางด้านวิทยาการคอมพิวเตอร์ในช่วง (1916 - 1975) ได้นำเสนอแนวคิด "การมีได้หลากหลายรูปแบบ (Polymorphism)" ในปี 1967 ได้กล่าวเกี่ยวกับ "Ad hoc Polymorphism" ไว้ดังนี้

"Ad-hoc polymorphism is obtained when a function works, or appears to work, on several different types (which may not exhibit a common structure) and may behave in unrelated ways for each type. Parametric polymorphism is obtained when a function works uniformly on a range of types; these types normally exhibit some common structure." – Strachey 1967

จากแนวคิดข้างต้นทำให้ Ad hoc Polymorphism ได้นำมาพัฒนาได้ 2 ประเภท
(1) Function Overloading และ (2) Operator Overloading

การมีหลากหลายรูปแบบ (Polymorphism)



Ad hoc Polymorphism - Operator Overloading

ในภาษาจาวาสามารถจัดการกับตัวดำเนินการ เช่น +, -, *, / ได้อย่างมีประสิทธิภาพเมื่อใช้กับประเภทข้อมูลที่แตกต่างกัน จาวาจะปรับการทำงานของตัวดำเนินการโดยอัตโนมัติให้ตรงกับประเภทข้อมูลที่ใช้งาน ซึ่งเป็นการแสดงถึงแนวคิดของ polymorphism แบบ Operator Overloading ตัวอย่างเช่น

```
String name = "Taravichet" + "Titijaroonroj";  
int num = 5 + 2;
```

Handwritten annotations: "Concat" with an arrow pointing to the "+" operator in the first line, and "sum" with an arrow pointing to the "+" operator in the second line. Above the strings, "String" is written twice. Above the integers, "int" is written twice.

หรือตัวอย่างถัดมา int และ float เป็นประเภทข้อมูลที่แตกต่างกัน ดังนั้นการใช้งานตัวดำเนินการในลักษณะที่แตกต่างกับประเภทข้อมูลเหล่านี้ก็สามารถถือว่าเป็นส่วนหนึ่งของแนวคิด polymorphism

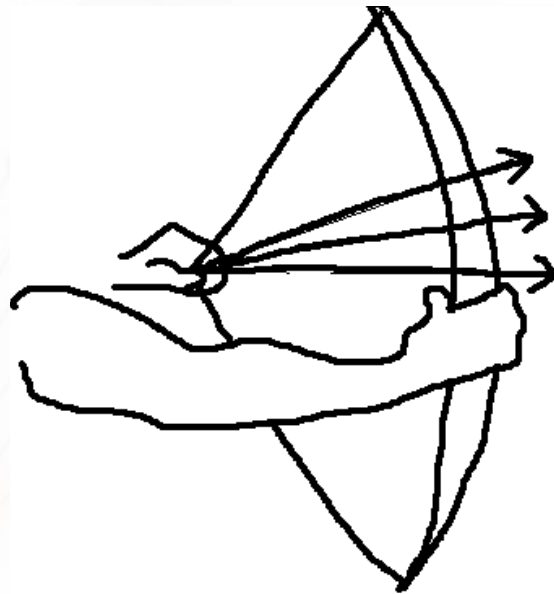
```
int i = 10 - 3;  
float f = 10.5 - 3.5;
```

Handwritten annotations: "int" is written above 10 and 3 in the first line, and "float" is written above 10.5 and 3.5 in the second line. Below 10 and 3, "double" is written. Below 10.5 and 3.5, "double" is written. A blue circle is drawn around the "-" operator in both lines.

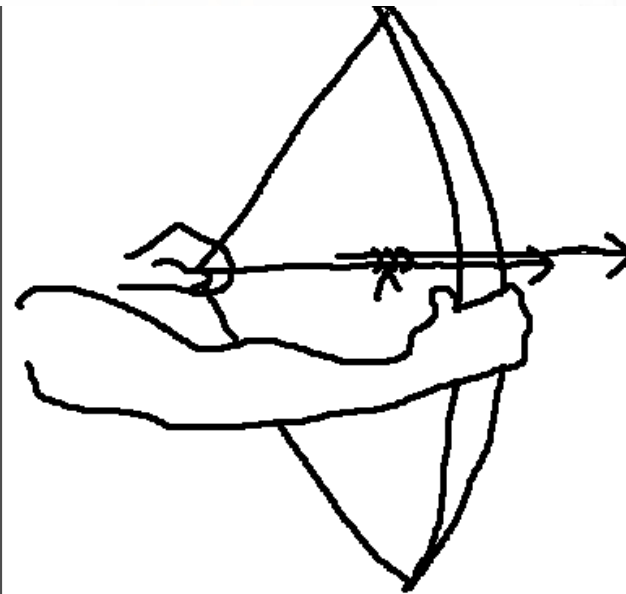
ดังนั้น สามารถสรุปได้ว่าภาษาจาวามีความสามารถในการ Operator Overloading แบบจำกัด คุณสมบัตินี้เป็นส่วนหนึ่งของภาษา และไม่อนุญาตให้นักพัฒนาทำการ Operator Overloading ด้วยตนเอง แต่ไม่มี Operator Overriding

Method Overloading และ Overriding

การทำ Method Overloading และ Overriding ทั้งสองล้วนเป็นตัวอย่างของ Ad hoc Polymorphism ในภาษาจาวาที่แสดงให้เห็นถึงการใช้ชื่อเมธอดเดียวกันในลักษณะที่แตกต่างกัน



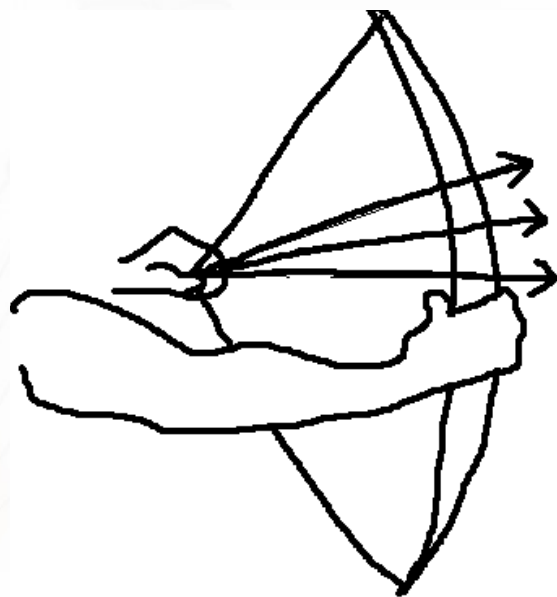
Overloading



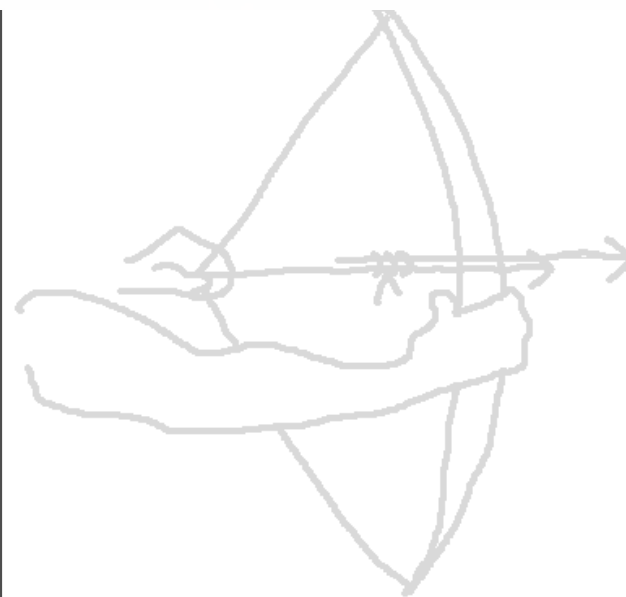
Overriding

Method **Overloading** และ Overriding

การทำ Method Overloading และ Overriding ทั้งสองล้วนเป็นตัวอย่างของ Ad hoc Polymorphism ในภาษาจาวาที่แสดงให้เห็นถึงการใช้ชื่อเมธอดเดียวกันในลักษณะที่แตกต่างกัน



Overloading



Overriding

Method Overloading



Woof

Speak();



Hae Hae

Speak();



เรียกใช้ method เดิม
แต่ป้อน input / parameter ต่างกัน
result อาจเหมือนหรือต่าง

Method Overloading

การ Method Overloading มีข้อกำหนดและลักษณะการทำงานดังนี้

- เมธอดที่มีชื่อเดียวกันภายในคลาสเดียวกัน
- เมธอดที่มีชื่อเดียวกัน แต่มีพารามิเตอร์ที่แตกต่างกัน เช่น จำนวน, ประเภท และลำดับพารามิเตอร์
- ไม่จำเป็นต้องมีประเภทของการคืนค่า (return) เหมือนกัน

```
public class Main {  
    public int add(int a, String b) {  
        return a + Integer.parseInt(b);  
    }  
    public double add(String a, int b) {  
        return b + Integer.parseInt(a);  
    }  
    public static void main(String[] args) {  
        Main m = new Main();  
        System.out.println(m.add(1, "2") + " and " + m.add("1", 2));  
    }  
}
```

① method ต้องชื่อเดียวกัน

② อยู่ภายใน class เดียวกัน /
class เดียวกัน

③ parameter ต้องต่างกัน
แบบใดแบบหนึ่ง

int str str int
mapping auto ตาม datatype ที่ parameter รับ

3 and 3.0

Method Overloading

การ Method Overloading ที่พบเจอได้ในภาษาจาวา อาทิเช่น คลาส `PrintStream` มีการ Method Overloading ของเมธอด `println()` ไว้ 9 รูปแบบ เพื่อรองรับอินพุต (input) อย่างหลากหลาย ส่งผลให้เมธอดมีความยืดหยุ่นในการใช้งานมากยิ่งขึ้น

<code>void</code>	<code><u>println</u>(boolean x)</code>	Prints a boolean and then terminate the line.
<code>void</code>	<code><u>println</u>(char x)</code>	Prints a character and then terminate the line.
<code>void</code>	<code><u>println</u>(char[] x)</code>	Prints an array of characters and then terminate the line.
<code>void</code>	<code><u>println</u>(double x)</code>	Prints a double and then terminate the line.
<code>void</code>	<code><u>println</u>(float x)</code>	Prints a float and then terminate the line.
<code>void</code>	<code><u>println</u>(int x)</code>	Prints an integer and then terminate the line.
<code>void</code>	<code><u>println</u>(long x)</code>	Prints a long and then terminate the line.
<code>void</code>	<code><u>println</u>(<u>Object</u> x)</code>	Prints an Object and then terminate the line.
<code>void</code>	<code><u>println</u>(<u>String</u> x)</code>	Prints a String and then terminate the line.

Method Overloading

ตัวอย่าง เมธอดที่มีชื่อเดียวกันแต่มีจำนวนพารามิเตอร์ที่แตกต่างกัน

```
public class MyMath {  
    public int minus(int a, int b) {  
        return (a-b);  
    }  
    public int minus(int a, int b, int c){  
        return (a-b-c);  
    }  
}  
public class Main {  
    public static void main(String[] args) {  
        MyMath m = new MyMath();  
        System.out.println(m.minus(10,5));  
        System.out.println(m.minus(10,2,5));  
    }  
}
```

Method Overloading

ตัวอย่าง เมธอดที่มีชื่อเดียวกันแต่มีประเภทพารามิเตอร์ที่แตกต่างกัน

```
public class MyMath {
    public double minus(double a, double b) {
        return (a-b);
    }
    public double minus(String a, String b){
        return (Double.parseDouble(a)- Double.parseDouble(b));
        // return minus(Double.parseDouble(a),Double.parseDouble(b));
    }
}

public class Main {
    public static void main(String[] args) {
        MyMath m = new MyMath();
        System.out.println(m.minus(10,5));
        System.out.println(m.minus("10","2"));
    }
}
```


Method Overloading ไม่สามารถทำได้โดยเปลี่ยนประเภทการคืนค่าของเมธอด กล่าวคือ การ Method Overloading ของเมธอด 2 เมธอดใดต้องมีพารามิเตอร์ต่างกัน ดังนั้น ถ้า 2 เมธอดใดที่มีพารามิเตอร์เดียวกัน และอยู่ภายในคลาสเดียวกัน จะก่อให้เกิดข้อผิดพลาดถึงแม้ว่าทั้งสองเมธอดมีการส่งคืนค่าที่แตกต่างกันก็ตาม

```
public class MyMath {  
    public int minus(int x, int y){  
        return x-y;  
    }  
    public double minus(int a, int b){  
        return (x-y);  
    }  
}
```

error parameter
ไม่ต่างกัน!

ดู datatype ไม่สนใจชื่อ variable

```
public class Main {  
    public static void main(String[] args) {  
        MyMath m = new MyMath();  
        System.out.println(m.minus(5,2));  
    }  
}
```

```
MyMath.java:5: error: me  
thod minus(int, int) is  
already defined in class  
Sample
```

Method Overloading

ตัวอย่าง การทำ Method Overloading ที่ถูกต้อง

```
public class Main {  
    public static void main(String[] args) {  
        Main m = new Main();  
        m.hi();      m.hi("Bank");      m.hi("Tara", "Vichet");  
    }  
    public void hi(){  
        System.out.println("Hi");  
    }  
    public void hi(String n){  
        System.out.println("Hi "+n);  
    }  
    public void hi(String f, String l){  
        System.out.println("Hi " + f + " " + l);  
    }  
}
```

ผลลัพธ์

Hi

Hi Bank

Hi Tara Vichet

Method Overloading

ตัวอย่าง การทำ Method Overloading ที่ ไม่ถูกต้อง

```
public class Main {  
    public static void main(String[] args) {  
        Main m = new Main();  
        m.hi();      m.hi("Bank");      m.hi("Tara", "Vichet");  
    }  
}
```

```
public void hi() {      System.out.println("Hi");    }  
public String hi() {    return "Hi";                }  
public void hi(String n) {  
    System.out.println("Hi "+n);  
}  
public void hi(String f) {  
    System.out.println("Hi " + f);  
}  
}
```

Handwritten annotations: "สับสน" (Confused) with arrows pointing to the first two `hi()` methods, and "ซ้ำ" (Repeat) with an arrow pointing to the third `hi()` method.

ผลลัพธ์

Main.java:11: error: method hi() is
already defined in class Main

```
public String hi() {  
    ^
```

Main.java:17: error: method hi(Strin
g) is already defined in class Main

```
public void hi(String f) {  
    ^
```

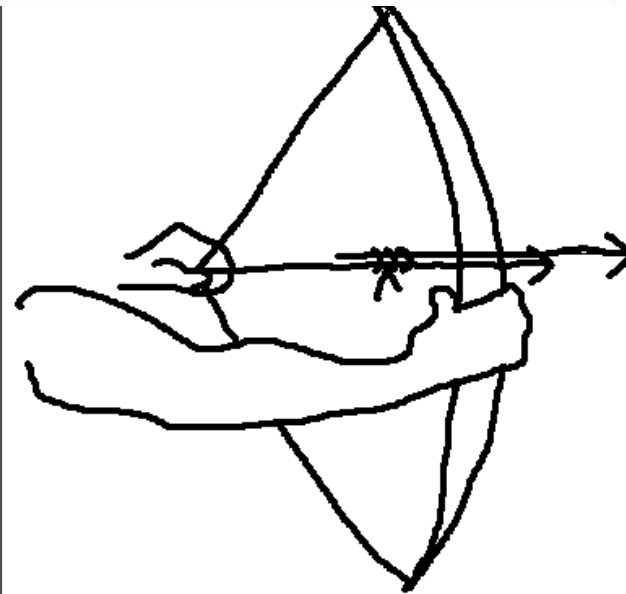
2 errors

Method Overloading และ Overriding

การทำ Method Overloading และ Overriding ทั้งสองล้วนเป็นตัวอย่างของ Ad hoc Polymorphism ในภาษาจาวาที่แสดงให้เห็นถึงการใช้ชื่อเมธอดเดียวกันในลักษณะที่แตกต่างกัน



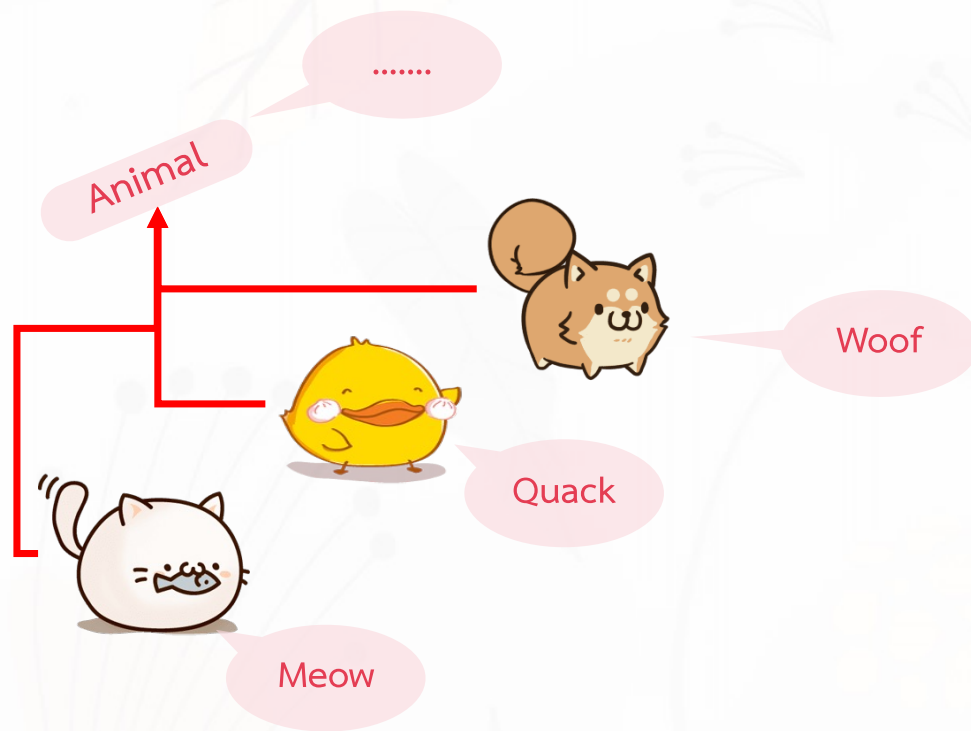
Overloading



Overriding

Method Overriding

(การต้งบอด)



① เพิ่มโค้ด

② เปลี่ยนงาน

Speak();



เป็นการอนุญาตให้คลาสลูก (Subclass) สามารถเพิ่มเติมการทำงานจำเพาะ (เพิ่มโค้ด) ในเมธอดที่คลาสแม่ (Superclass) มีไว้แล้วได้ โดยมีเงื่อนไขดังนี้

Method Overriding

การ Method Overriding มีข้อกำหนดและลักษณะการทำงานดังนี้

- เมธอดที่มี Header Method เหมือนกันเกิดขึ้นระหว่างคลาสแม่และคลาสลูก
- คลาสลูก Overriding เมธอดจากคลาสแม่ โดยมี Header Method เดียวกัน กล่าวคือ ต้องมี ชื่อเมธอด พารามิเตอร์ และประเภทของค่าที่คืนกลับมาเหมือนกัน ยกเว้น Access Modifier ที่ใหญ่กว่าเดิมได้
- การ Overriding เมธอดใน คลาสลูก มักจะมีการทำงานที่แตกต่างจากคลาสแม่
- ใช้ keyword @Override ในภาษาจาวา เพื่อระบุว่าเมธอดนั้นถูก Overriding

↳ ใส่ / ไม่ใส่ก็ได้

Method Overriding

```
public class1 Animal {2  
    public void1 sound()3 {  
        System.out.println("Animal sound");  
    }  
}  
public class Dog extends Animal {  
    @Override1  
    public void1 sound()3 {  
        System.out.println("Bark");  
    }  
}  
public class Main {  
    public static void main(String[] args) {  
        new Animal().sound();  
        new Dog().sound();  
    }  
}
```

1 2 3 must be same!

Animal sound
Bark

Method Overriding

```
public class Animal {  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}  
public class Dog extends Animal {  
    @Override  
    public void sound() {  
        System.out.println("Bark");  
    }  
}  
public class Main {  
    public static void main(String[] args) {  
        new Animal().sound();  
        new Dog().sound();  
    }  
}
```

default

↑

Animal sound

Animal sound
Bark

Method Overriding

```
public class Animal {  
    public void sound() {  
        System.out.println("Animal sound");  
    }  
}  
  
public class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Bark");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        new Animal().sound();  
        new Dog().sound();  
    }  
}
```

Handwritten notes:
- A bracket on the left groups the `Animal` and `Dog` classes.
- Next to the `Animal` class: `{scope}`
- Next to the `Dog` class: `!error!`
- Next to the `void sound()` in `Dog`: `default`

```
error: sound() in Dog cannot  
override sound() in Animal  
    void sound() {  
        ^  
        attempting to assign weaker  
        access privileges; was public  
1 error
```


Method Overriding

```
public class Student { class ใหม่
    protected String id;
    protected String name; method
    public void printDetail() {
        System.out.println("ID " + id);
        System.out.println("Name " + name);
    }
}

public class GradStudent extends Student {
    protected String thesisTitle; attribute ใหม่
    @Override
    public void printDetail() {
        System.out.println("ID " + id);
        System.out.println("Name " + name);
        System.out.println("Title " + thesisTitle);
    }
}
```

being attribute and print

ใช้ id ของพ่อแม่

super.printDetail();

พิมพ์ของพ่อแม่

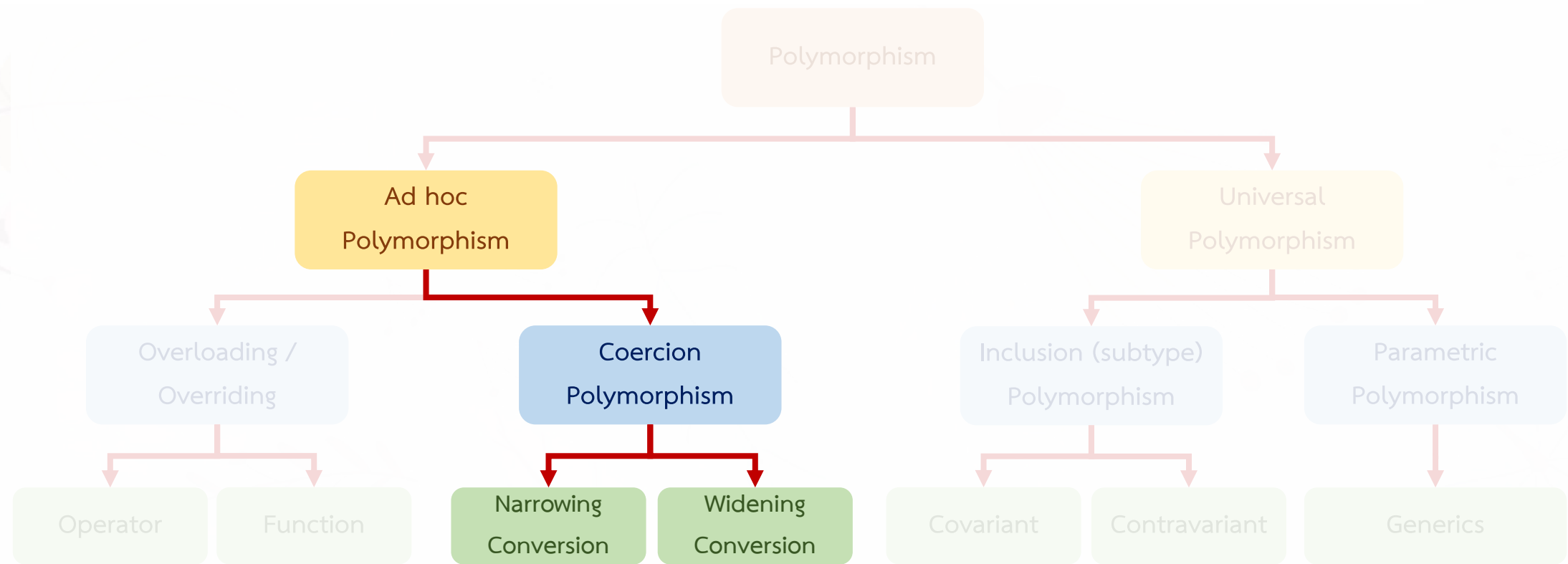
Overriding แบบ extend คือสามารถ override method

Method Overriding

→ เพื่อกำหนด rate of interest ที่ต่างกัน

```
public class Bank๖๖ น  
    public double getRateOfInterest(){return 0;}  
}  
  
public class SCB extends Bank๒, ๓  
    public double getRateOfInterest(){return 2.5;}  
}  
  
public class KTB extends Bank๒, ๓  
    public double getRateOfInterest(){return 1.75;}  
}  
  
public class KBANK extends Bank๒, ๓  
    public double getRateOfInterest(){return 2.90;}  
}
```

การมีหลากหลายรูปแบบ (Polymorphism)



Static Binding or Dynamic Binding

Coercion Polymorphism

Coercion Polymorphism หมายถึง กระบวนการที่ compiler หรือ Interpreter ทำการแปลงประเภทข้อมูลหนึ่งไปเป็นประเภทอื่นโดยอัตโนมัติ โดยที่นักพัฒนาไม่จำเป็นต้องระบุการแปลงนั้นโดยตรงในโค้ด ซึ่งภาษาจาวารองรับการแปลงประเภทข้อมูลในลักษณะที่ขยาย (Widening) ขอบเขตของข้อมูล เช่น จาก int ไปเป็น float แต่ไม่รองรับการแปลงที่ย่อ (Narrowing) ขอบเขตของข้อมูล เช่น จาก double ไปเป็น int

```
public class Temperature {  
    public double fToC (double fahrenheit) {  
        return (fahrenheit - 32)*5/9; Δ  
    }  
  
    public static void main(String args[]) {  
        Temperature temp = new Temperature();  
        System.out.print(temp.fToC(98));  
    }  
}
```

Widening
(ก.แปลงแบบขยาย)
จาก int → double
บันทึกไว้ได้
// แปลง int เป็น double อัตโนมัติ

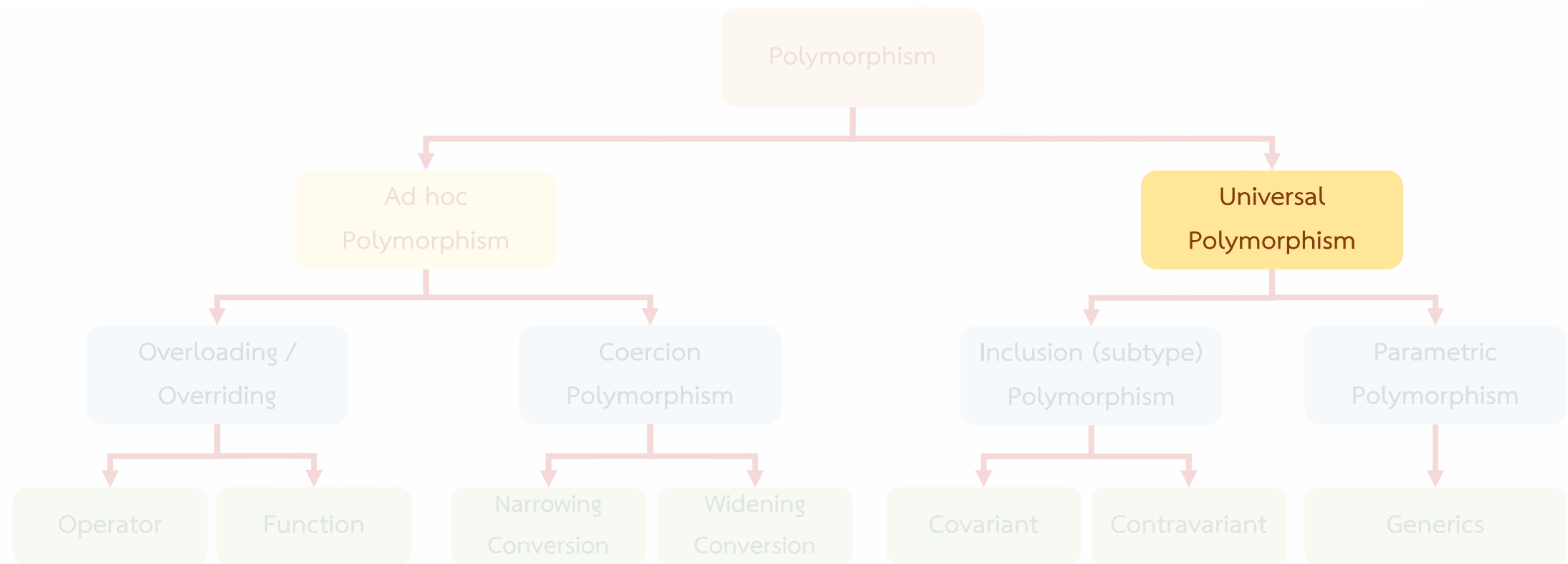
Coercion Polymorphism

Coercion Polymorphism หมายถึง กระบวนการที่ compiler หรือ Interpreter ทำการแปลงประเภทข้อมูลหนึ่งไปเป็นประเภทอื่นโดยอัตโนมัติ โดยที่นักพัฒนาไม่จำเป็นต้องระบุการแปลงนั้นโดยตรงในโค้ด ซึ่งภาษาจาวารองรับการแปลงประเภทข้อมูลในลักษณะที่ขยาย (Widening) ขอบเขตของข้อมูล เช่น จาก int ไปเป็น float แต่ไม่รองรับการแปลงที่ย่อ (Narrowing) ขอบเขตของข้อมูล เช่น จาก double ไปเป็น int

```
public class Temperature {  
    public double fToC (int fahrenheit) {  
        return (fahrenheit - 32)*5/9;  
    }  
  
    public static void main(String args[]) {  
        Temperature temp = new Temperature();  
        System.out.print(temp.fToC (98.4)); // ข้อผิดพลาด เพราะ Narrowing Conversion  
    }  
}
```

double → int ไม่ได้

การมีหลากหลายรูปแบบ (Polymorphism)

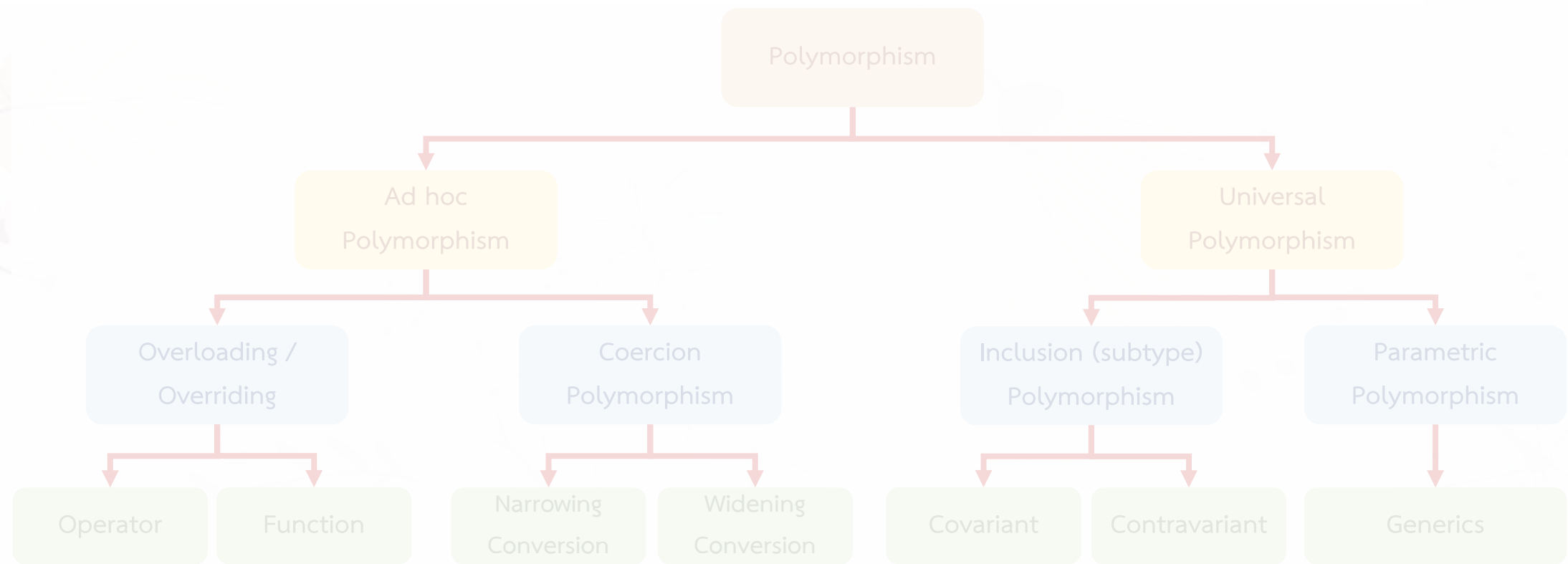


Static Binding or Dynamic Binding

Universal Polymorphism

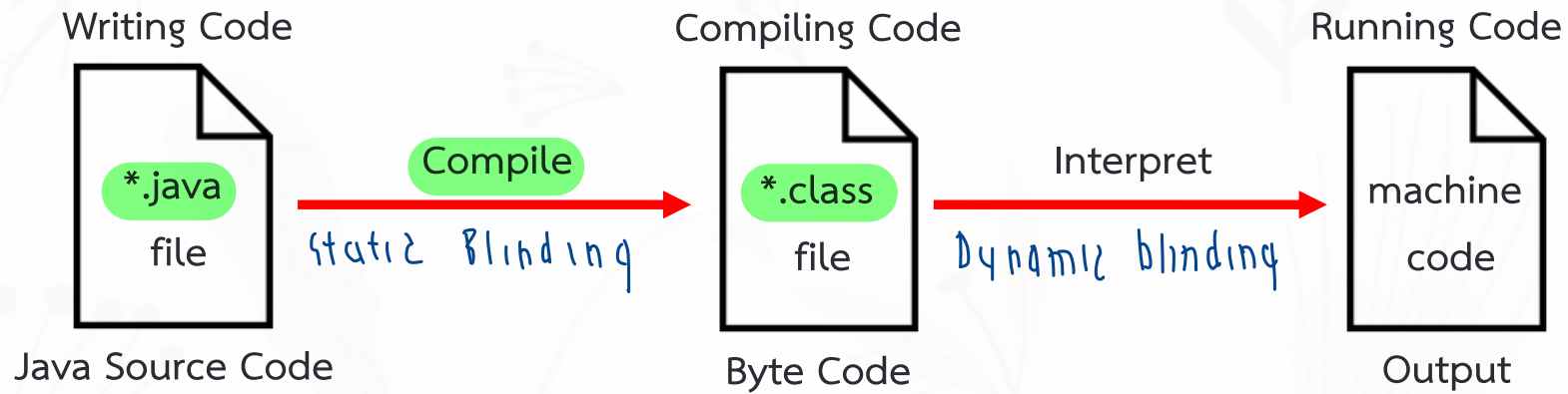
Universal polymorphism อ้างถึงความสามารถของโปรแกรมในการจัดการกับประเภทข้อมูลต่างๆ อย่างทั่วถึงผ่านโครงสร้างเทมเพลตที่เป็นสากล ซึ่งไม่ขึ้นกับประเภทของข้อมูลที่ใช้งาน มันประกอบไปด้วย 2 ประเภท ได้แก่ (1) Inclusion Polymorphism และ (2) Parametric Polymorphism

การมีหลากหลายรูปแบบ (Polymorphism)



Static Binding or Dynamic Binding

Static Binding และ Dynamic Binding



Static และ Dynamic Binding เป็นส่วนสำคัญในการเขียนโปรแกรมแบบ Static binding ให้ประสิทธิภาพที่ดีในขณะคอมไพล์ ในขณะที่ Dynamic binding มอบความยืดหยุ่นและช่วยให้โปรแกรมมีความสามารถในการตอบสนองต่อสถานการณ์ที่แตกต่างกันในรันไทม์ ซึ่งทั้ง Static และ Dynamic Binding มีความสัมพันธ์กันดังนี้

Static Binding และ Dynamic Binding

Object/

- **Static Binding** หมายถึงการที่ **Java Compiler** ตัดสินใจเลือกเมธอดที่จะเรียกใช้ขณะที่โค้ดกำลังถูกคอมไพล์ ไม่ใช่ขณะรันไทม์ Static binding มักเกี่ยวข้องกับ method overloading และ access ของตัวแปรที่เป็น final, private หรือ static

Object/

- **Dynamic Binding** หมายถึงการที่ **Java Runtime Environment (JRE)** เลือกเมธอดที่จะเรียกใช้ขณะที่โปรแกรมกำลังรัน ขึ้นอยู่กับประเภทของอ็อบเจกต์จริงที่อ้างอิงโดยตัวแปร Dynamic binding มักเกี่ยวข้องกับ method overriding ในการสืบทอดคลาส.

Static Binding และ Dynamic Binding

Static Binding เป็นการผูกมัดระหว่างเมธอดกับอ็อบเจกต์ที่เกิดขึ้นในขณะ compile time ซึ่งหมายความว่า **Compiler จะตัดสินใจเรียกใช้เมธอดใดในขณะที่กำลังคอมไพล์โค้ดเท่านั้น** ตัวอย่างเช่น การ Method Overloading เนื่องจาก Compiler ตัดสินใจเลือกเมธอดที่เหมาะสมตามประเภทของพารามิเตอร์ที่ส่งไป

```
public class Calculator {  
    public int add(int a, int b) { return a + b; }  
    public double add(double a, double b) { return a + b; }  
  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
        // Compiler ตัดสินใจเรียกใช้ add(int, int)  
        System.out.println(c.add(5, 10));  
        // Compiler ตัดสินใจเรียกใช้ add(double, double)  
        System.out.println(c.add(5.0, 10.0));  
    }  
}
```

Static Binding และ Dynamic Binding

Dynamic Binding เป็นการผูกมัดระหว่างเมธอดกับอ็อบเจกต์ที่เกิดขึ้นในขณะ runtime โดย Java Runtime จะตัดสินใจว่าจะเรียกใช้เมธอดใดขึ้นอยู่กับประเภทของอ็อบเจกต์จริง ตัวอย่าง การ Method Overriding เนื่องจาก Java Runtime เลือกใช้เมธอดที่ถูก override ในคลาสลูก

```
public class Animal {
    public void sound() { System.out.println("Animal sound"); }
}
public class Dog extends Animal {
    public void sound() { System.out.println("Bark"); }
}
public class Main {
    public static void main(String[] args) {
        Animal myAnimal = new Dog();
        myAnimal.sound();
        // ผลลัพธ์จะเป็น "Bark" เนื่องจากเลือกใช้ method sound ของ Dog ใน runtime
    }
}
```


Static Binding และ Dynamic Binding

เป็นหลักการของการสร้างวัตถุของคลาสที่มีการสืบทอดกันได้หลากหลายรูปแบบ ตัวอย่างการประกาศให้วัตถุของคลาสใด ๆ Superclass obj; ซึ่งสามารถสร้างได้ 2 แบบ ได้แก่

แบบที่ 1 ประกาศและสร้างอ็อบเจกต์จากคลาสเดียวกัน

ประกาศ สร้าง
`Superclass obj = new Superclass();`

แบบที่ 2 ประกาศจาก Superclass และสร้างอ็อบเจกต์จาก Subclass

ประกาศแม่ สร้างลูก
`Superclass obj = new Subclass();`

Static Binding และ Dynamic Binding

```
public class ๖๖๗Student {...} ลูก Student
public class GradStudent extends Student {...} ลูก GradStudent
public class PhDStudent extends GradStudent {...}

public class Main {
    public static void main(String[] args) {
        // ตัวอย่าง การประกาศ Student แล้วสร้างวัตถุเป็น Student หรือ GradStudent หรือ PhDStudent
        Student stu1 = new Student();
        Student stu1 = new GradStudent(); # ประกาศแบบสแตติก → ผลงานได้ผล
        Student stu1 = new PhDStudent();

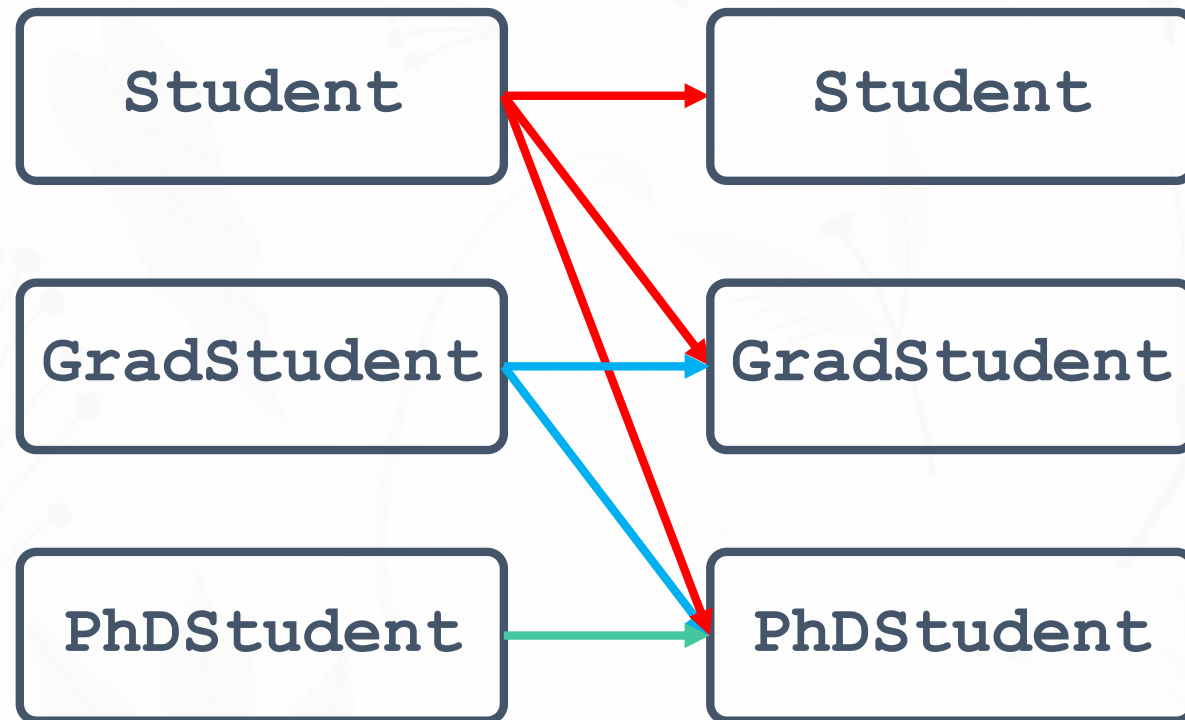
        // เช่นเดียวกับ การประกาศ GradStudent แล้วสร้างวัตถุเป็น GradStudent หรือ PhDStudent
        GradStudent stu2 = new GradStudent();
        GradStudent stu2 = new PhDStudent(); # ประกาศแบบไดนามิก → ผลงานได้

        // อย่างไรก็ตาม ภาษาจาวาไม่สามารถประกาศ Subclass แต่สร้างวัตถุเป็น Superclass ได้
    }
}
```

Static Binding และ Dynamic Binding

(1) ประกาศ

(2) สร้าง




```
public class Student{
    protected String id, name;
    public void printInfo(Student s) {
        System.out.println("ID "+s.id+ "\nName "+s.name);
    }
}
public class GradStudent extends Student{
    protected String thesisTitle;
}
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student();
        GradStudent g1 = new GradStudent();

        // นักศึกษาสามารถที่จะเรียกใช้เมธอด printInfo(Student s) ได้หลายรูปแบบดังนี้
        s1.printInfo(s2);
        s1.printInfo(g1);
    }
}
```

นอกจากนี้ แนวคิดของ Dynamic Binding สามารถนำไปประยุกต์ใช้กับการส่งผ่าน argument ได้หลายรูปแบบ ด้วยเช่น ในกรณีที่เมธอดมี argument เป็นข้อมูลชนิดคลาส นักศึกษาสามารถส่งอ็อบเจกต์ของคลาสที่เป็น subclass ของคลาสนั้นแทนได้ ตัวอย่างเช่น

Static Binding และ Dynamic Binding

```
public class Fruit {  
    public int getEnergy() { return 0; }  
}  
public class Apple extends Fruit {  
    public int getEnergy() {  
        return 10;  
    }  
}  
public class Orange extends Fruit {  
    public int getEnergy() {  
        return 15;  
    }  
}  
public class Banana extends Fruit {  
    public int getEnergy() {  
        return 20;  
    }  
}
```

Static Binding และ Dynamic Binding

```
public class Dog {  
    private int energy;  // ๒๒ ๒' ๒' = ๒๒ ๒' ๒'  
    public int getEnergy() { return energy; }  
    public void setEnergy(int energy) { this.energy = energy; }  
    public void eat(Fruit f) { this.setEnergy(this.getEnergy() + f.getEnergy()); }  
}  
public class Main {  
    public static void main(String[] args) {  
        Dog d1 = new Dog();  
        Apple a = new Apple();  
        Orange o = new Orange();  
        Banana b = new Banana();  
        System.out.println("Dog's Energy is " + d1.getEnergy());  
        System.out.println("Dog's Energy is " + d1.getEnergy());  
        System.out.println("Dog's Energy is " + d1.getEnergy());  
        System.out.println("Dog's Energy is " + d1.getEnergy() + " after ate " + b.getName());  
    }  
}
```

Diagram illustrating the execution flow for the `eat` method call in the `Main` class:

```
d1.eat(a);  
d1.eat(o);  
d1.eat(b);
```

A curved arrow points from the `d1.eat(b);` line in the `main` method to the corresponding line in the diagram.

Dynamic Binding - Virtual Method Invocation

VM

ตอนที่ 1

```
public class Vehicle{  
    public void move(){  
        System.out.println("Move in Vehicle");  
    }  
}
```

```
public class Car extends Vehicle{  
    public void move(){  
        System.out.println("Move in Car");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Vehicle v1 = new Car();  
        v1.move();  
        Car c1 = new Car();  
        c1.move();  
    }  
}
```

Dynamic Binding

สนใจว่าส่วไหน ไม่ใช้ประกาศ

ผลลัพธ์

Move in Car

Move in Car

ทั้ง 2 ตัวสร้างมาจากรุ่น Car

Dynamic Binding - Virtual Method Invocation

ตอนที่ 2

```
public class Vehicle {๖  
    public void move()๗{      System.out.println("Move in Vehicle"); }  
}  
  
public class Car extends Vehicle{  
    public void move(){      System.out.println("Move in Car"); }  
    public void move2()๘{      System.out.println("Move in Car"); }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Vehicle v1 = new Car();  
        v1.move2();  
        Car c1 = new Car();  
        c1.move2();  
    }  
}
```

ผลลัพธ์

```
Main.java:4: error: cannot find symbol
    v1.move2();
        ^
    2
```

```
symbol: method move2()  
location: variable v1 of type Vehicle  
1 error
```

Dynamic Binding - Virtual Method Invocation

โปรแกรมภาษาจาวาพิจารณาเรียกใช้เมธอดจากชนิดของอ็อบเจกต์ที่สร้างขึ้น ตัวอย่างเช่น

```
Vehicle v1 = new Car();  
v1.move();
```

เป็นคำสั่งที่เรียกใช้เมธอด move2() ของคลาส Car ไม่ใช่เมธอดของคลาส Vehicle แต่คอมไพเลอร์ของภาษาจาวาจะไม่อนุญาตให้เรียกใช้เมธอดใด ๆ ก็ตามที่ไม่มีการประกาศอยู่ในเมธอดของ superclass ที่กำหนดไว้ ตัวอย่างเช่น

```
Vehicle v1 = new Car();  
v1.move2();
```


Dynamic Binding - Virtual Method Invocation

เพื่อจะแก้ไขปัญหข้างต้น การ Casting Object จึงถูกนำมาใช้งานสำหรับกรณีนี้

```
(ClassName) ObjectName
```

ตัวอย่างเช่น

```
Vehicle v1 = new Car();  
(Car) v1).move2(); # แปลงป้าบป๊ะกาศ
```

↳ ส่งผลค่าบ่น่งนั้นบ่น่า้น

Dynamic Binding - Virtual Method Invocation

ตอนที่ 3

```
public class Vehicle{
    public void move(){        System.out.println("Move in Vehicle");    }
}

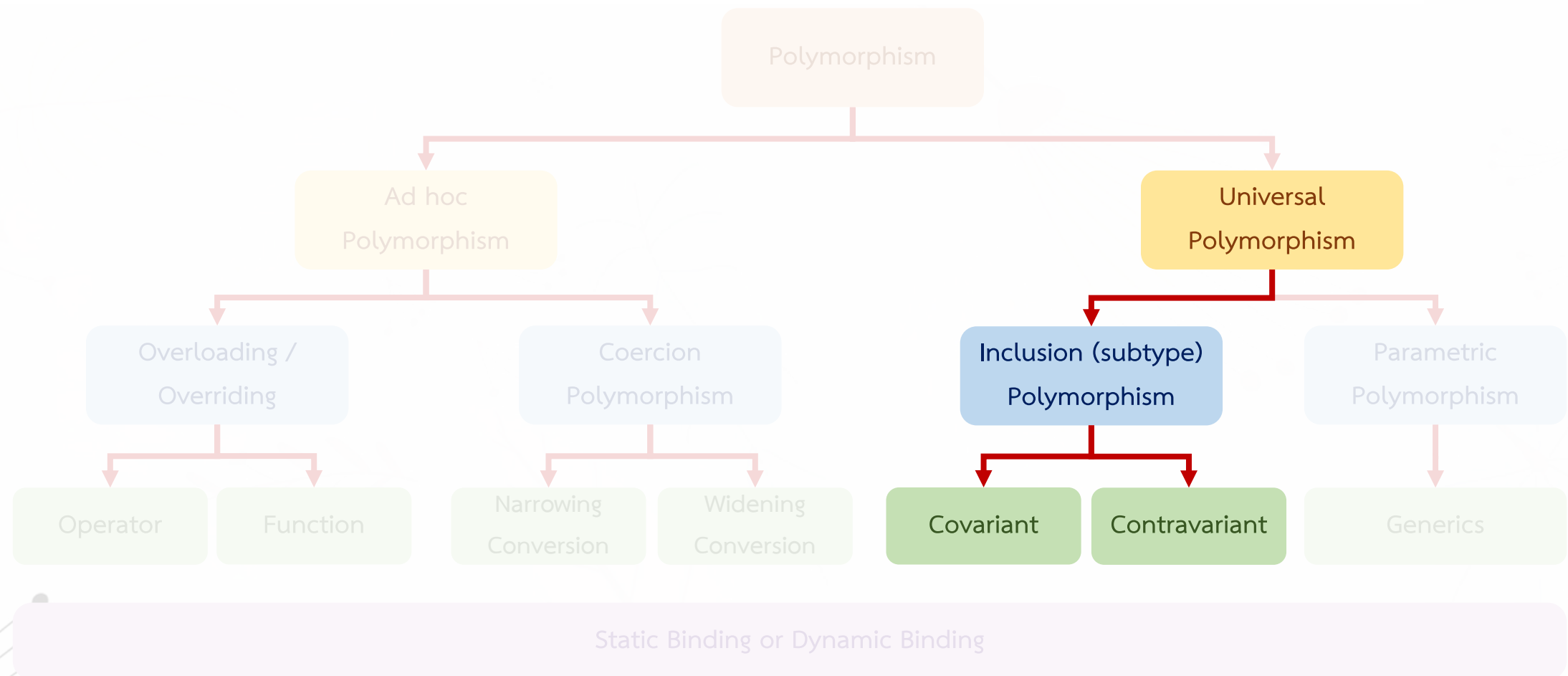
public class Car extends Vehicle{
    public void move(){        System.out.println("Move in Car");        }
    public void move2(){        System.out.println("Move in Car");        }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v1 = new Car();
        ((Car)v1).move2(); # สังเกตได้จากรหัสนี้แหละ ***
        Car c1 = new Car();
        c1.move2();
    }
}
```

ผลลัพธ์

Move in Car
Move in Car

การมีหลากหลายรูปแบบ (Polymorphism)



Universal Polymorphism - Inclusion Polymorphism

Inclusion polymorphism (หรือที่เรียกว่า subtype polymorphism) หมายถึง สถานการณ์ที่อ็อบเจกต์ของคลาสลูก (subclass) สามารถถูกจัดการหรืออ้างอิงผ่านอ็อบเจกต์ของคลาสแม่ (superclass) หรืออินเตอร์เฟส (Interface) ซึ่งสามารถจัดการได้ 2 รูปแบบ ได้แก่ แนวคิดของ Covariant และ Contravariant ที่เกี่ยวข้องกับวิธีที่ประเภทข้อมูลสามารถถูกแทนที่หรือแปลงได้ในการสืบทอด

Universal Polymorphism - Inclusion Polymorphism

แนวคิดของ Covariant ในทางการเขียนโปรแกรม คือ ความสามารถในการเปลี่ยนประเภทข้อมูลตามการสืบทอด สิ่งนี้เกี่ยวข้องกับการที่ subtype (คลาสลูก) สามารถใช้แทน supertype (คลาสแม่) ได้ ซึ่ง Covariant นำมาใช้ในหลายบริบทในการเขียนโปรแกรมแบบ Object-Oriented และมักเกี่ยวข้องกับการ Inheritance และ Polymorphism

```
public class Animal {}  
public class Dog extends Animal {}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal myAnimal = new Dog(); // Covariant Dog is a subtype of Animal  
    }  
}
```

ประกาศคลาสลูก

ในตัวอย่าง covariant ที่เกิดขึ้นในการสืบทอดคลาสจะพบว่า Dog เป็น subtype ที่สามารถถูกอ้างอิงผ่าน Animal (supertype) ได้

Universal Polymorphism - Inclusion Polymorphism

แนวคิดของ Contravariant (ใช้กับ Generics): เป็นการเขียนโปรแกรมที่สามารถใช้ประเภทข้อมูลที่เป็น supertype (คลาสแม่) แทนที่ subtype (คลาสลูก) ได้ กล่าวคือ หากมีการสืบทอดระหว่างคลาสแล้ว อ็อบเจกต์ของ supertype สามารถอ้างอิงถึงคลาสที่เป็น subtype ได้

```
public class Animal {}
public class Dog extends Animal {}
public class Main {
    public void processAnimals(List<? super Dog> animals) {
        // สามารถใส่ข้อมูลของ Dog หรือ super subtype ของ Dog ลงใน List
    }
    public static void main(String[] args) {
        List<Animal> myAnimals = new ArrayList<>();
        new Main().processAnimals(myAnimals);
        // Contravariant: List of Animal สามารถใช้สำหรับข้อมูลของ Dog
    }
}
```

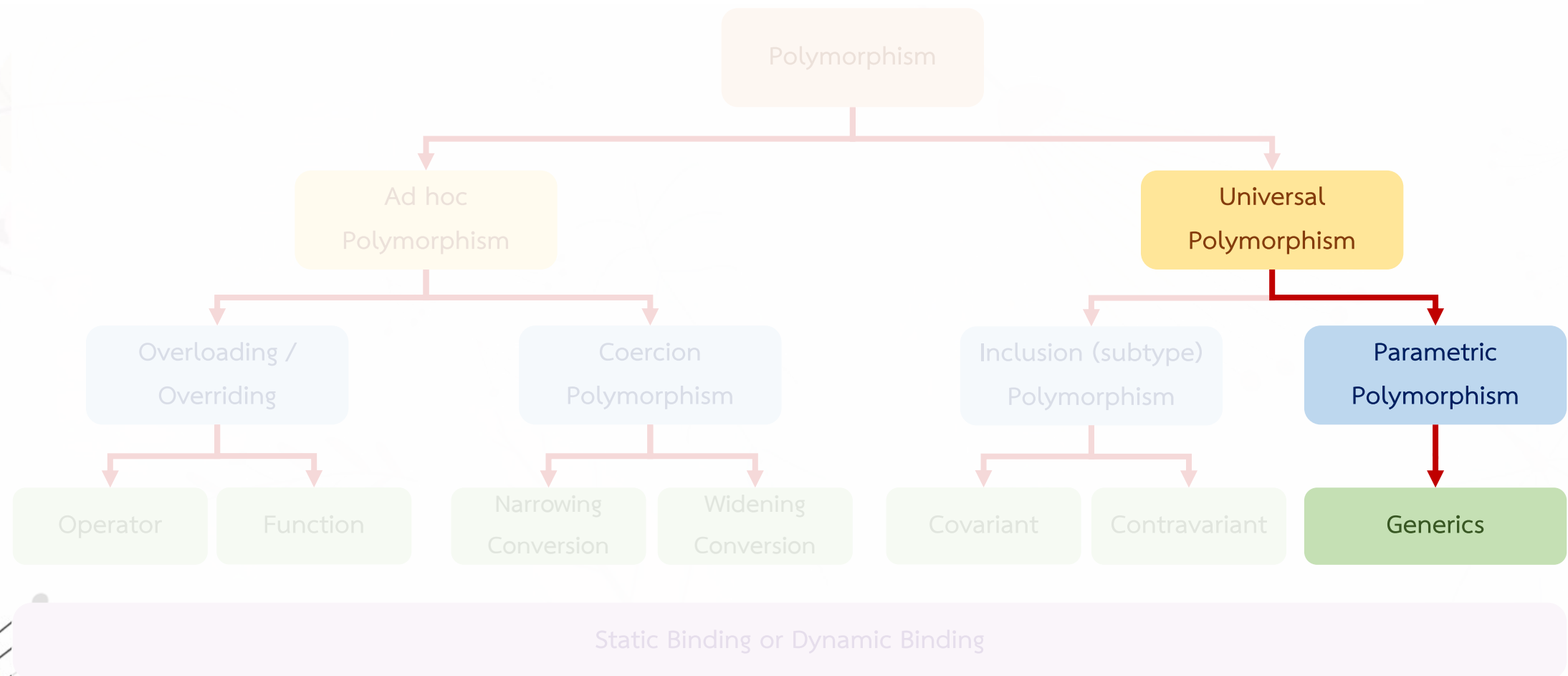


```
public class Animal {}
public class Dog extends Animal {}
public class Main {
    public void processAnimals(List<Animal> animals) {
        // สามารถใส่ข้อมูลของ Dog หรือ subtype ของ Dog ลงใน List
    }
    public static void main(String[] args) {
        List<Animal> myAnimals = new ArrayList<>();
        new Main().processAnimals(myAnimals);
    }
}
```

การใช้ List<? super Dog> และ List<Animal> ใน method processAnimals มีความแตกต่างกันดังนี้

- List<? super Dog> เป็นการใช้ wildcards แบบ contravariant ใน Java Generics หมายถึง List ของ Dog หรือ supertype ของ Dog เช่น Animal หรือ Object ข้อดี คือ ช่วยเพิ่มความยืดหยุ่นในการรับ List ของประเภทข้อมูลต่างๆ ที่เป็น supertype ของ Dog
 - List<Animal> เป็นการระบุประเภทข้อมูลอย่างชัดเจน หมายถึง List ของ Animal เท่านั้น แม้ว่า Dog เป็น subclass ของ Animal แต่การใช้ List<Animal> จำกัดเฉพาะ List ที่มีอ็อบเจกต์เป็น Animal เท่านั้น ไม่สามารถใช้กับ List ของ supertype อื่นๆ ของ Dog ได้
- การใช้ List<? super Dog> จึงให้ความยืดหยุ่นมากกว่าในบางสถานการณ์ เช่น เมื่อต้องการให้ method นี้สามารถรับ List ของประเภทข้อมูลใดๆ ที่สามารถจัดเก็บ Dog ได้ ไม่จำกัดเฉพาะแค่ Animal

การมีหลากหลายรูปแบบ (Polymorphism)



Parametric polymorphism



(Photo: courtesy of AlanTuring.net)

Christopher Strachey นักวิจัยทางด้านวิทยาการคอมพิวเตอร์ในช่วง (1916 - 1975) ได้นำเสนอแนวคิด "การมีได้หลากหลายรูปแบบ (Polymorphism)" ในปี 1967 ได้กล่าวเกี่ยวกับ "Ad hoc Polymorphism" ไว้ดังนี้

"Ad-hoc polymorphism is obtained when a function works or appears to work, on several different types (which may not exhibit a common structure) and may behave in unrelated ways for each type. *Parametric polymorphism is obtained when a function works uniformly on a range of types; these types normally exhibit some common structure.*" – Strachey 1967

จากแนวคิดข้างต้นทำให้ Ad hoc Polymorphism ได้นำมาพัฒนาได้ 2 ประเภท
(1) Function Overloading และ (2) Operator Overloading

Universal Polymorphism - Parametric polymorphism

ในภาษาจาวา Parametric Polymorphism หมายถึง ความสามารถในการกำหนดเมธอดและประเภทข้อมูลในแบบทั่วไป ที่ทำงานได้ตามพารามิเตอร์ที่ส่งมาในขณะ runtime โดยไม่ลดทอนความปลอดภัยของประเภทข้อมูล เพื่อสร้างโครงสร้าง และเมธอดที่ทำงานได้กับประเภทข้อมูลต่างๆ โดยพฤติกรรมของโครงสร้างหรือเมธอดจะขึ้นอยู่กับประเภทข้อมูลที่ส่งมาใน ขณะที่โครงสร้างหรือเมธอดนั้นถูกสร้างหรือเรียกใช้งาน

```
public class GenericBox<T> {  
    private T content;  
    public void setContent(T content) {  
        this.content = content;  
    }  
    public T getContent() {  
        return content;  
    }  
    public static void main(String[] args) {  
        GenericBox<String> stringBox = new GenericBox<>();  
        stringBox.setContent("Hello");  
        System.out.println("String Value: " + stringBox.getContent());  
    }  
}
```

Handwritten notes in Thai:
- Above the first <T>: 4tr (generic type)
- Next to the first <T>: ยังไม่ระบุว่าใช้เก็บอะไร (haven't specified what it stores yet)
- Above the second <T>: 4tr (generic type)

Universal Polymorphism - Parametric polymorphism

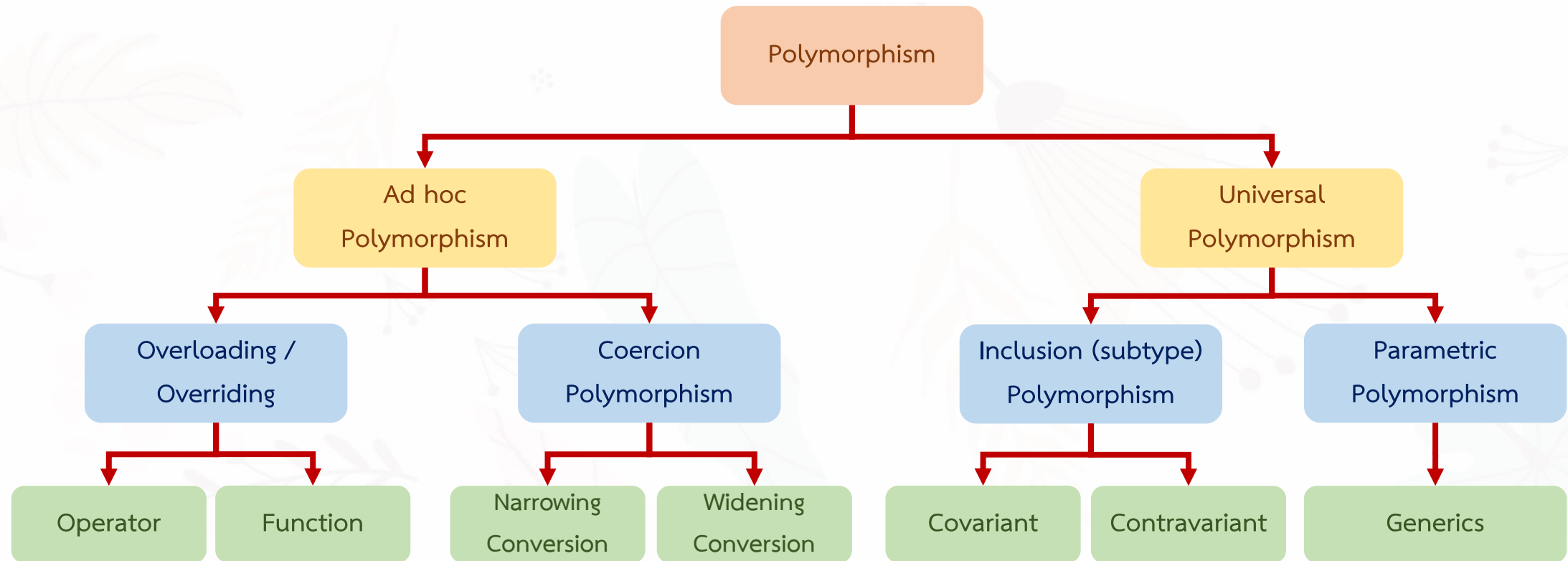
```
public class GenericBox<T> {  
    private T content;  
    public void setContent(T content) {  
        this.content = content;  
    }  
    public T getContent() {  
        return content;  
    }  
    public static void main(String[] args) {  
        GenericBox<String> stringBox = new GenericBox<>();  
        stringBox.setContent("Hello");  
        System.out.println("String Value: " + stringBox.getContent());  
    }  
}
```

↑ Datatype อ้างอิงเท่านั้น

ตัวอย่างนี้ GenericBox เป็นคลาสที่ใช้ Generics เพื่อสร้างอ็อบเจกต์ที่สามารถจัดเก็บข้อมูลประเภทต่างๆ ได้ แต่ข้อควรระวัง

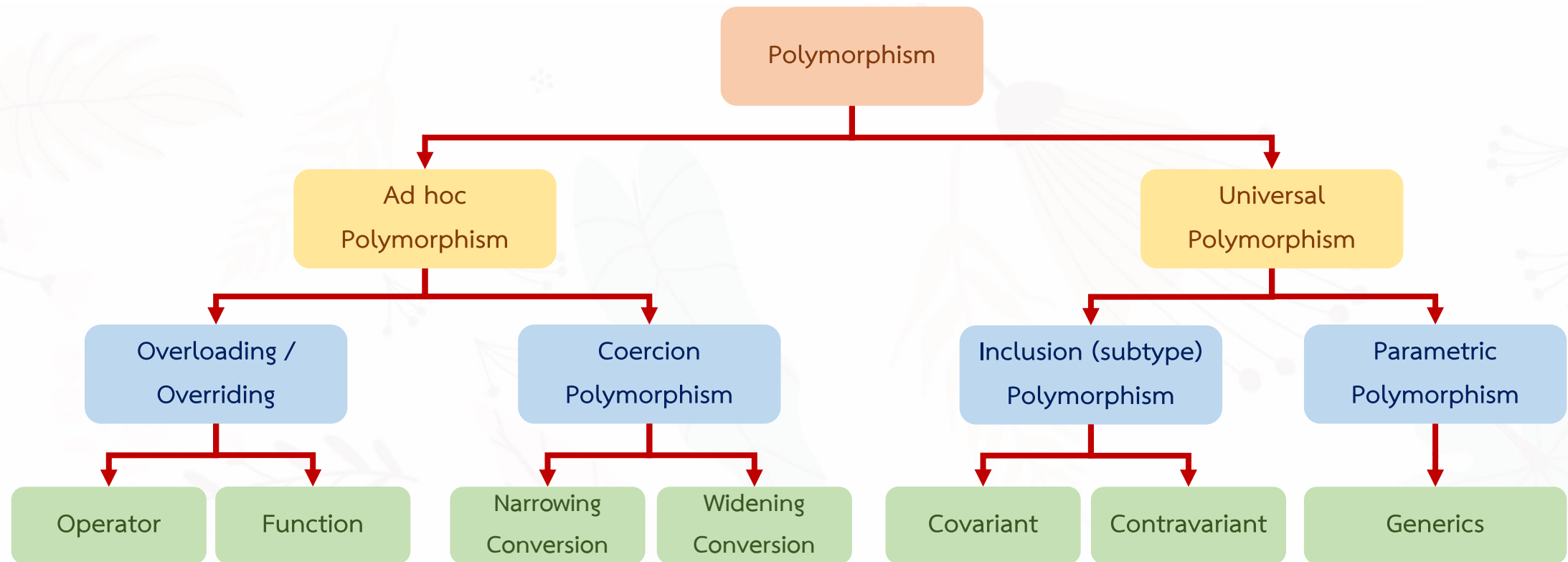
- การกำหนด Generic จะอาศัยเครื่องหมาย <...> ภายในเครื่องหมายจะใส่ตัวอักษรหรือคำอะไร
- การประกาศอ็อบเจกต์จะสามารถระบุชนิดข้อมูลใน <...> ได้แต่เพียงตัวแปรชนิดอ้างอิงเท่านั้น

การมีหลากหลายรูปแบบ (Polymorphism)



Static Binding or Dynamic Binding

การมีหลากหลายรูปแบบ (Polymorphism)



Static Binding or Dynamic Binding

คลาสที่ชื่อ Object

ภาษาจาวาได้กำหนดให้คลาสใดๆ สามารถจะสืบทอดคลาสอื่นได้เพียงคลาสเดียวเท่านั้น ดังนั้น คลาสทุกคลาสในภาษาจาวา ถ้าไม่ได้สืบทอดจากคลาสใดเลยจะถือว่าสืบทอดจากคลาสที่ชื่อ Object

ตัวอย่างเช่น

```
public class Student * {  
    . . .  
}  
  
public class Student extends Object {  
    . . .  
}
```

คลาสที่ชื่อ Object จะมีเมธอดที่สำคัญคือ

public String toString() และ public boolean equals(Object o)

เมธอด toString()

เมธอด toString() ในภาษาจาวาเป็นเมธอดที่ใช้สำหรับแปลงอ็อบเจกต์เป็นสตริง (String) โดยปกติแล้วเมื่อเราพิมพ์หรือแสดงอ็อบเจกต์ด้วย System.out.println() จาวาจะเรียกใช้เมธอด toString() ของอ็อบเจกต์เพื่อแปลงเป็นสตริงโดยอัตโนมัติ เมธอด toString() ถูกกำหนดในคลาส Object ตามค่าเริ่มต้น, toString() จะแสดงชื่อคลาสของอ็อบเจกต์ตามด้วย "@" และแฮชโค้ดของอ็อบเจกต์ อาทิ เช่น MyClass@1a2b3c

```
public class Car {  
    private String brand;  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println(new Car()); ①  
        //System.out.println(new Car().toString()); ②  
    }  
}
```

ไม่ทำงาน

ผลลัพธ์

Car@1c20c684

เมธอด toString()

แต่นักศึกษาสามารถ Override เมธอดนี้ในคลาสที่สร้างขึ้นเอง เพื่อให้แสดงข้อมูลที่มีความหมายและสอดคล้องกับความต้องการมากขึ้น

```
public class Car {  
    private String brand;  
    @Override  
    public String toString() {  
        return "Car [brand=" + brand + "]";  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        System.out.println(new Car());  
        //System.out.println(new Car().toString());  
    }  
}
```

ผลลัพธ์

Car [brand=null]

เมธอด equals(Object o)

เมธอด equals(Object o) ในภาษาจาวาเป็นเมธอดที่ใช้เพื่อเปรียบเทียบว่าสองอ็อบเจกต์มีค่าเท่ากันหรือไม่ ค่าเริ่มต้นของเมธอดนี้ถูกกำหนดในคลาส Object และจะเปรียบเทียบตำแหน่งในหน่วยความจำของอ็อบเจกต์ อาทิ เช่น มันจะเท่ากันก็ต่อเมื่อมันเป็นอ็อบเจกต์เดียวกัน

```
public class Car {  
    private String brand;  
    public void setBrand(String brand) { this.brand = brand; }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car c1 = new Car();  
        c1.setBrand("BMW");  
        Car c2 = new Car();  
        c2.setBrand("BMW");  
        System.out.println(c1.equals(c2));  
    }  
}
```

ผลลัพธ์

false

เมธอด equals(Object o)

อย่างไรก็ตาม นักศึกษาสามารถ Override หรือ Overload เมธอด equals() เพื่อให้สามารถเปรียบเทียบเนื้อหาหรือสถานะภายในของอ็อบเจกต์ได้ ซึ่งสามารถทำได้โดยตรวจสอบค่าภายในอ็อบเจกต์เพื่อดูว่าเท่ากันหรือไม่

```
public class Car {  
    private String brand;  
    public void setBrand(String brand) { this.brand = brand; }  
    public boolean equals(Object o){  
        return this.brand == ((Car)o).brand;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car c1 = new Car();      Car c2 = new Car();  
        c1.setBrand("BMW");      c2.setBrand("BMW");  
        System.out.println(c1.equals(c2));  
    }  
}
```

ผลลัพธ์

true

เมธอด equals(Object o)

อย่างไรก็ตาม นักศึกษาสามารถ Override หรือ Overload เมธอด equals() เพื่อให้สามารถเปรียบเทียบเนื้อหาหรือสถานะภายในของอ็อบเจกต์ได้ ซึ่งสามารถทำได้โดยตรวจสอบค่าภายในอ็อบเจกต์เพื่อดูว่าเท่ากันหรือไม่

```
public class Car {  
    private String brand;  
    public void setBrand(String brand) { this.brand = brand; }  
    public boolean equals(Car o){  
        return this.brand == o.brand;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car c1 = new Car();      Car c2 = new Car();  
        c1.setBrand("BMW");      c2.setBrand("BMW");  
        System.out.println(c1.equals(c2));  
    }  
}
```

ผลลัพธ์

true



THANKS !